Testable Phase 2 E2E Automation

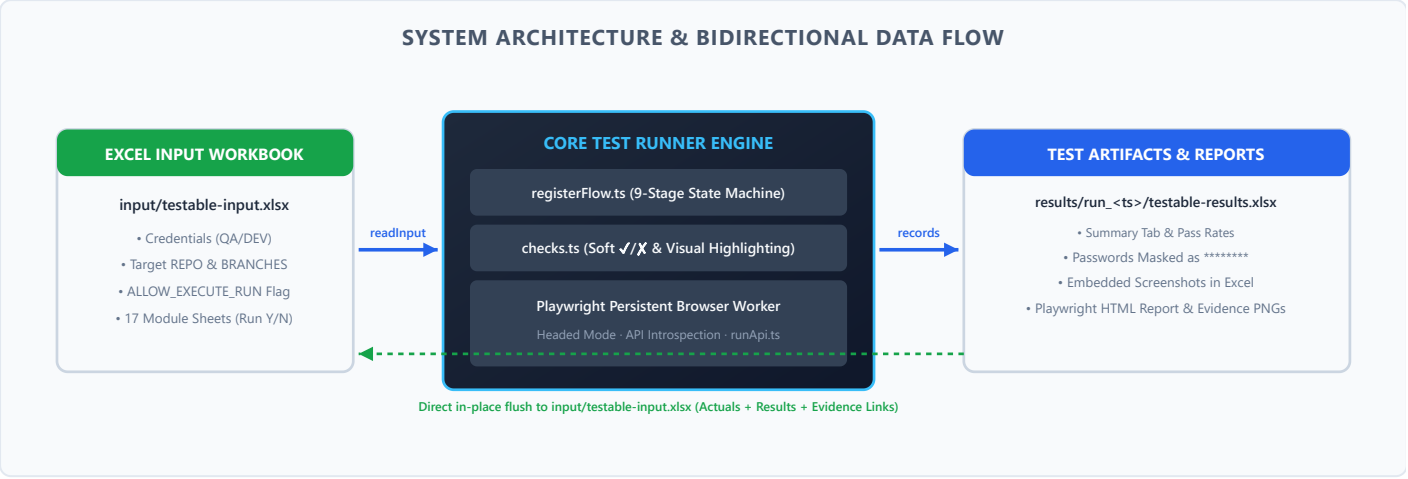
Complete User Manual, Installation Guide & Pipeline Workflow Architecture

ENGINE	DATA DRIVER	TARGET PLATFORM	STANDARD
Playwright + TypeScript	Excel (testable-input.xlsx)	Testable Cloud (QA / DEV)	Automated E2E QA Specification

1 Executive Overview & Core Architecture

The **Testable Phase 2 E2E Test Suite** is an enterprise-grade, modular, Excel-driven end-to-end automation framework built for the **Testable Platform** (SaaS whitebox testing, code intelligence, and confidence scoring).

Unlike conventional UI automation suites that launch isolated browser contexts per test, this framework executes a **unified, serialized user journey across 17 distinct modules** within a single persistent browser worker, maintaining session state across all screens.



2 Things to Install & Setup Commands

To run this suite from scratch, perform these three installation steps in your terminal:

2.1 Install Node.js Dependencies

```
npm install
```

Packages Installed: `@playwright/test` (Playwright test harness), `typescript` & `ts-node` (TypeScript execution), `exceljs` (Excel styling and image embedding), `xlsx` (Workbook reading), and `@types/node`.

2.2 Download Playwright Chromium Browser Binaries

```
npx playwright install chromium
```

> **Browser Binary Path:** Chromium installs to `%LOCALAPPDATA%\ms-playwright\chromium-<version>\chrome-win64\chrome.exe`. If this step is omitted, tests fail with: `Executable doesn't exist at chrome.exe`.

2.3 Install VS Code / Antigravity Testing Extension (For Manual UI Testing)

1. Press `Ctrl + Shift + X` (Extensions Marketplace).
2. Search for: `ms-playwright.playwright` (Playwright Test for VS Code by Microsoft).
3. Click **Install**.
4. The **Testing Beaker Icon (🧪)** will appear in the left Activity Bar with the full interactive test tree.

3 Complete Command Reference

Command	Action Performed	When & How to Use
`npm run template`	Creates `input/testable-input.xlsx` from scratch.	Run once on a fresh machine. Refuses to overwrite existing files to protect saved passwords. Add `-- --force` to recreate.
`npm run sync`	Synchronizes existing Excel workbook with code.	Run after pulling new git updates. Safely adds new test case rows while keeping your existing passwords untouched.
`npm test` *or* `npm run test:flow`	Executes the complete 17-module serialized journey.	Main E2E command. Signs in, connects repository, executes pipeline, and audits all screens in one continuous run.
`npm run test:branches`	Executes test matrix across multiple Git branches.	Spawns sequential runs for every branch specified in the `BRANCHES` key. Creates separate results folders per branch.
`npx playwright test --ui`	Launches Playwright Interactive Desktop GUI.	Best for manual visual debugging. Allows running individual test cases, DOM inspection, and time-travel tracing.
`npm run report`	Opens Playwright HTML Report in the browser.	Run after any test execution to inspect visual step execution logs and traces.

4 Configuration & Credentials Setup

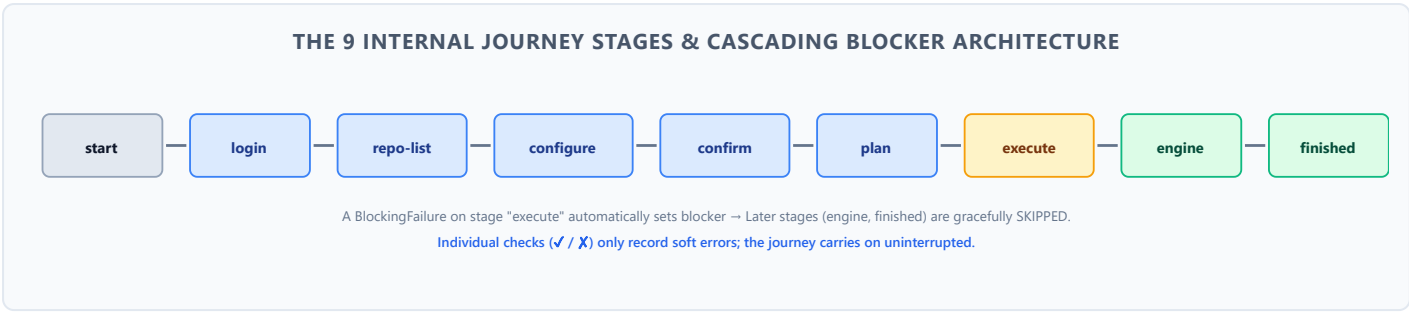
All configuration resides in one file: `input/testable-input.xlsx` on the **Input** sheet:

Key Name	Default / Example	Description & Impact
`ENV`	`QA`	Selects `QA` or `DEV` environment blocks.
`QA_BASE_URL`	`https://qa-frontend.testable.cc`	Frontend base URL for the QA deployment.
`QA_EMAIL`	*your-email@example.com*	Your normal test account email for QA login.
`QA_PASSWORD`	*your-password*	Account password (masked as `********` in results).
`HEADLESS`	`N`	`N` = Opens real browser window so you see the test live; `Y` = Runs invisibly in the background.
`SLOW_MO_MS`	`0` (Set to `300` - `500`)	Slows down browser actions by X ms so you can follow clicks and typing visually.
`REPO`	`VISVANTHA/CE-Platform-Stack`	The target repository to import, select, and analyze.
`BRANCHES`	`main,feature-branch`	Comma-separated branches. Provide at least 2 branches if testing the branch switcher.
`ALLOW_EXECUTE_RUN`	`N` (Change to `Y`)	`Y` = Clicks "Execute run" (starts real whitebox pipeline); `N` = Stops at plan stage to save execution credits.
`ALLOW_DUMMY_COMMIT`	`N`	`Y` = Clones repo, pushes a dummy README commit, and validates "Analyze latest commit" flow.

> **Critical Golden Rule:** Always **save and close** `testable-input.xlsx` in Microsoft Excel before running any test. Windows file locking will prevent the runner from writing test results back into the sheet.

5 Pipeline Workflow & State Machine

The runner utilizes an ordered state machine (`registerFlow.ts`). If an early milestone fails catastrophically, subsequent dependent tests are gracefully marked `SKIPPED ("Blocked by <id>")` rather than cascading into false failures.



6 Step-by-Step Walkthrough of the 17 Modules

#	Module Name	Stage Needed	Operations Performed & Assertions Made
1	**Login Module**	`start`	Validates pre-auth UI elements, performs credentialed login, verifies authentication cookies, and confirms post-login landing on `/execution-assistant` .
2	**Assistant Home**	`login`	Validates Execution Assistant homepage, headline greeting, analysis cards, composer slash command menu, and header elements.
3	**Repo Connection**	`login`	Opens "Change repository / branch", chooses configured REPO from GitHub list, selects primary branch, ticks "On PR merge", and clicks Confirm.
4	**Coverage Preview**	`confirm`	Validates language detection, signal profile breakdown, test readiness cards, and advances through "Continue".
5	**Run Plan**	`plan`	Verifies generated Run plan, task checklist, taxonomy execution matrix, and configuration summary.
6	**Context Module**	`plan`	Inspects the Repository ready status bar, Source details drawer (commit SHA, branch availability), and configuration change history.
7	**Execution Pipeline**	`plan`	Triggered if `ALLOW_EXECUTE_RUN = Y` . Clicks "Execute run", monitors the live 8-stage pipeline progression (Queued → Publishing), and waits for completion (~15-20 min).
8	**Confidence Engine**	`finished`	Validates Confidence Score (/100), Verdict (Pass/Hold/Block), Whitebox score breakdown, tool execution cards, taxonomy gates, and tab drilldowns.
9	**Confidence Graph**	`finished`	Audits interactive SVG flow graph, historical score trend lines, node connections, and pipeline metrics.
10	**Confidence Object**	`finished`	Validates organization-level aggregations, repository risk summaries, open risks, and status pills.
11	**Latest Commit**	`finished`	If enabled, clones repository, pushes a dummy README commit, clicks "Analyze latest commit", and verifies incremental analysis.
12	**Runs Dashboard**	`login`	Inspects Runs Dashboard, historical table rows, pagination, branch filter dropdowns, and search controls.
13	**Account & Billing**	`login`	Checks user profile, organization details, token quotas, and Stripe checkout entry points (without performing payments).
14	**Support & Chat**	`login`	Tests Help Center menu, Customer Support chat widget triggers, ticket modal, and User Guide links without spamming support.
15	**Redirects & Security**	`login`	Tests route protection, 404 handlers, legacy address redirects, SCM integration paths, and invalid run ID shielding.

#	Module Name	Stage Needed	Operations Performed & Assertions Made
16	**Sign Out**	`login`	Opens sidebar user menu, clicks Sign Out, confirms alert dialog, and verifies session cookies are cleared.
17	**Sign Up Module**	`start`	Tests form validation and error handling on `/auth/register` without creating phantom user accounts.

7 Visual Evidence & Reporting Deliverables

Every test run automatically produces deliverables in two locations:

7.1 Timestamped Run Deliverable Folder: `results/run_<timestamp>/`

- `testable-results.xlsx` : Fully formatted Excel report containing a dedicated **Summary Tab** (pass rates per module), with all passwords/secrets masked as `********` , and embedded screenshot thumbnails.
- `evidence/*.png` : Full-resolution PNG screenshots. Every captured image features a **green outline** (passing check) or **red outline** (failed check), plus a bottom banner stating the exact Case ID and step.
- `playwright-report/index.html` : Official Playwright interactive visual report.

7.2 In-Place Update to: `input/testable-input.xlsx`

When `WRITE_RESULTS_TO_INPUT = Y` , the module sheets in your local input file (`Login` , `Repo` , etc.) are directly populated with the latest **Actual Result** text (✓/✗), **Result** (PASS/FAIL/SKIPPED), and screenshot links.

8 Troubleshooting & FAQ

Symptom / Error	Root Cause	Exact Solution
`'ts-node' is not recognized`	`node_modules` dependencies not installed yet.	Run `npm install` in your terminal from the project root directory.
`Executable doesn't exist at ...\chrome.exe`	Playwright Chromium browser binaries missing.	Run `npx playwright install chromium` to download the required binaries.
`Could not replace input/testable-input.xlsx - is it open in Excel?`	Windows file lock prevents writing while Excel has the file open.	Save and close Microsoft Excel before running `npm test` .
Downstream tests marked `SKIPPED (Blocked by EXE-001)`	`ALLOW_EXECUTE_RUN` is set to `N` .	Change `ALLOW_EXECUTE_RUN` to `Y` on the `Input` sheet to permit running real pipelines.
Runs Dashboard shows `422` or 0 runs	Configured repository/branch has never run an analysis before.	Perform at least one full pipeline run (`ALLOW_EXECUTE_RUN = Y`). The Dashboard will automatically populate.
Testing icon (🧪) not visible in editor sidebar	Playwright extension not installed or icon hidden.	Install `ms-playwright.playwright` via Extensions (Ctrl+Shift+X), or launch `npx playwright test --ui` .